

Aula 1

Grandes Modelos de Linguagem

Gabriel Lenz Balatka

Orientador: Rafael Stubs Parpinelli

Joinville, 10 de junho de 2026



UDESC
UNIVERSIDADE
DO ESTADO DE
SANTA CATARINA

LABICOM
Laboratório de Pesquisa
em Inteligência Computacional

Sumário

- Contexto histórico e motivação
- Visão geral da arquitetura transformer
- Diferentes arquiteturas derivadas
- Visão geral da arquitetura decoder-only
- Estudo de algoritmo básico de tokenização (Byte-Pair Encoding)

Introdução

- Grandes Modelos de Linguagem (LLMs) estão cada vez mais populares e são utilizados por uma grande quantidade de usuários

Rumo ao bi: ChatGPT chega a 900 milhões de usuários ativos por semana

Recorde de acessos no chatbot de inteligência artificial foi anunciado junto de novos investimentos na OpenAI.



Nilton Cesar Monastier Kleina



27/02/2026, às 15:45



(KLEINA, 2026)

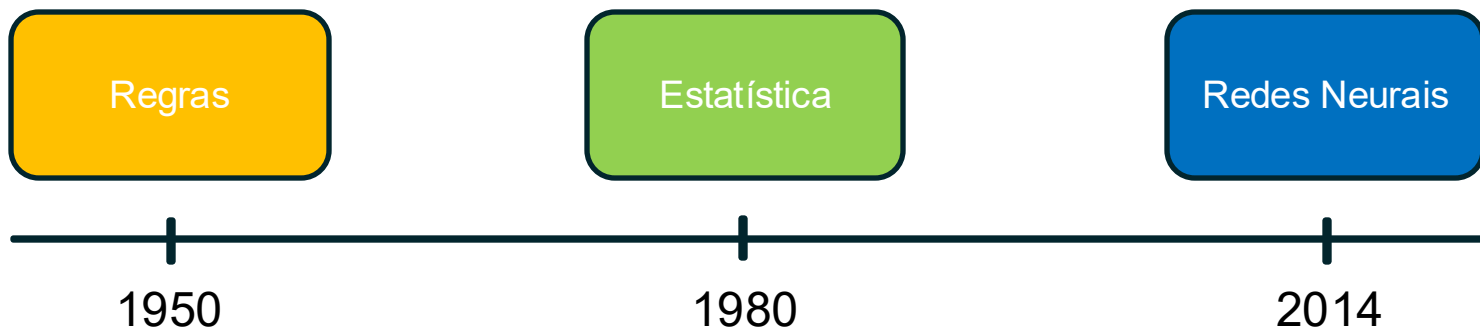
Famílias de LLMs

- GPT (OpenAI)
 - Claude (Anthropic)
 - Gemini (Google)
 - LLama (Meta)
 - Grok (xAI)
-
- Todas as famílias (tipos) de LLMs listadas são baseadas em uma arquitetura de rede neural chamada de Transformer

Início com a Tradução de Texto

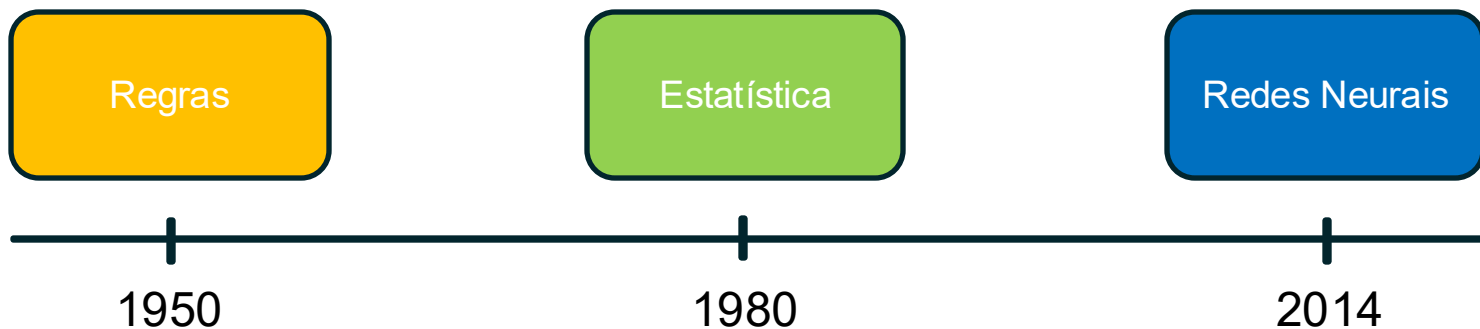
- A arquitetura Transformer foi publicada no artigo 'Attention is all you need' em 2017
- O objetivo original da arquitetura era de traduzir texto e posteriormente foi adaptado para geração de texto
- Tradução automática (*Machine Translation*)
 - Inglês: The quick brown fox jumps over the lazy dog
 - Português: A rápida raposa marrom pula sobre o cachorro preguiçoso

Tradução de Texto



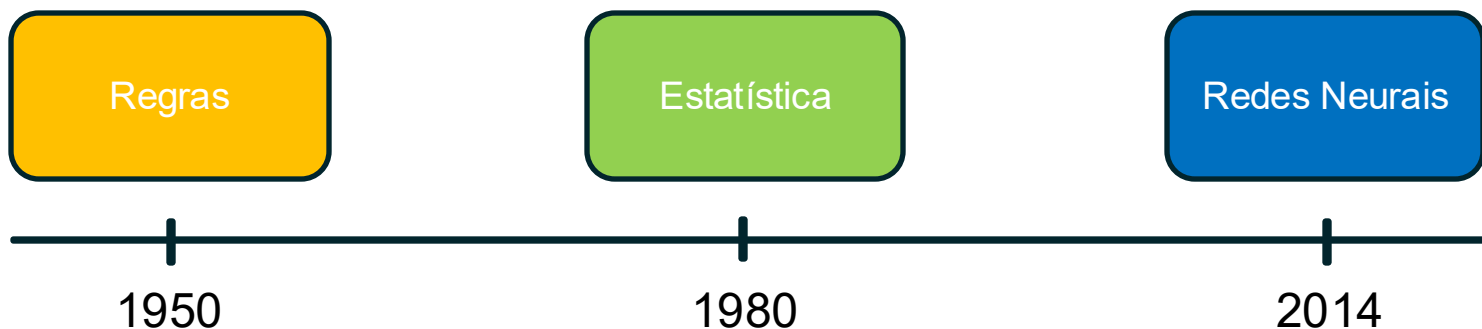
- Regras: Sistemas baseados em regras linguísticas e gramáticas elaboradas à mão
- Qual a principal limitação?

Tradução de Texto



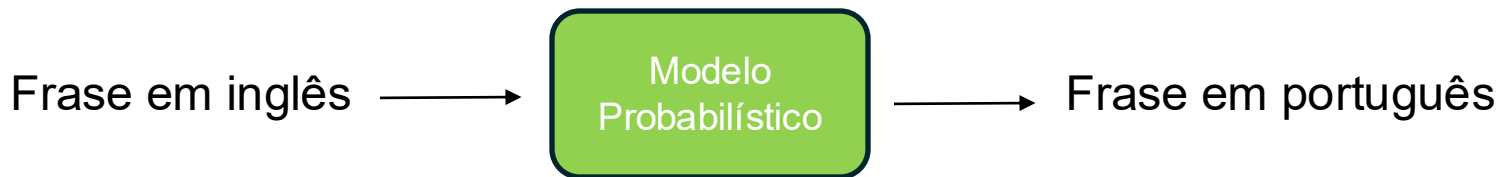
- Regras: Sistemas baseados em regras linguísticas e gramáticas elaboradas à mão
- Qual a principal limitação?
Pouco Escalável

Tradução de Texto



- Estatística: Treinamento de modelos probabilísticos a partir de dados
- Principal Limitação: Dificuldade em capturar dependências sintáticas e contexto longo

Treinamento Supervisionado



Inglês	Português
The quick brown fox jumps over the lazy dog	A rápida raposa marrom pula sobre o cachorro preguiçoso
The cat sat on the mat	O gato sentou no tapete
The sky is blue and the grass is green	O céu é azul e a grama é verde
...	...

- Milhares de pares entrada-saída são utilizados no treinamento

Tradução de Texto

- Até 2017, o estado da arte da tradução de texto era baseado em redes neurais recorrentes que processam o texto palavra por palavra
- Primeira estratégia de atenção foi proposta por (BAHDANAU, 2014), mas conexão entre termos distantes ainda era limitada
- Principais Limitações: Treinamento lento e dificuldade de tradução de textos longos

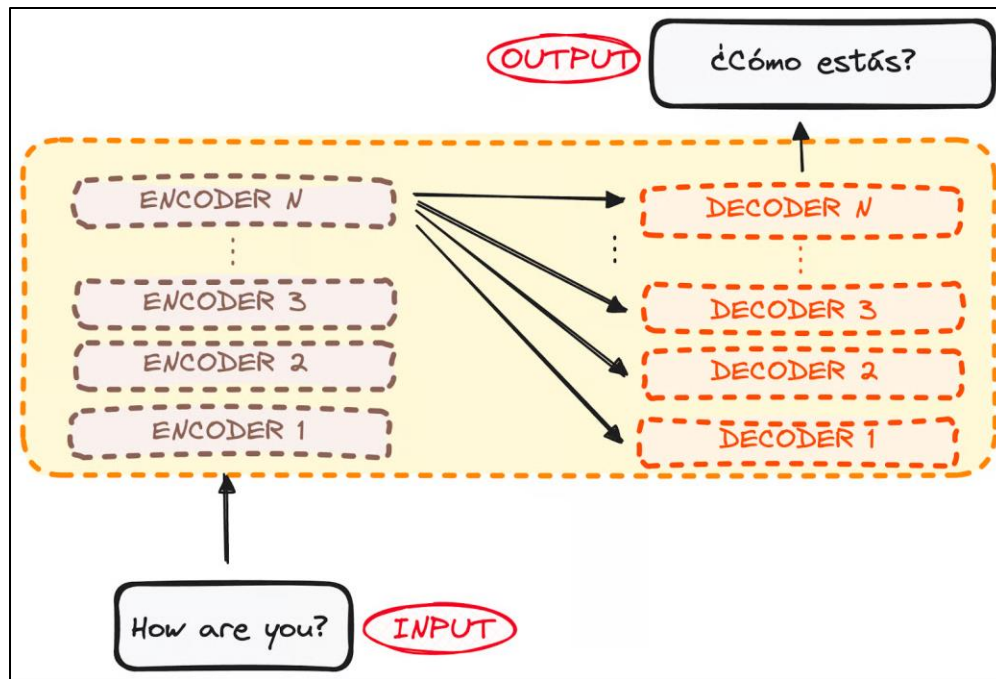
Arquitetura Transformer

- Com a arquitetura Transformer, duas melhorias são atingidas:
 - Conectar palavras distantes, independentemente da distância entre ambas
 - Possibilitar o treinamento em paralelo (Resultados mais significativos em menos tempo)

Arquitetura Transformer

- A arquitetura Transformer é composta por dois componentes principais: Encoder e Decoder
- O encoder é responsável por transformar a entrada em uma representação contextualizada
- O decoder utiliza a representação contextualizada para gerar texto novo

Arquitectura Transformer



(FERRER, 2026)

Modelos de Linguagem Pré-Treinados

- Através da arquitetura Transformer, é possível treinar modelos genéricos com grandes volumes de dados
- Com base na utilização dos componentes da arquitetura, pode-se classificar os modelos em três categorias:
 - Encoder-only
 - Decoder-only
 - Encoder-Decoder

Modelos de Linguagem Pré-Treinados

- Encoder-only
 - O exemplo mais conhecido é o BERT (Bidirectional Encoder Representations from Transformers)
 - Ótimo para entendimento e classificação de texto
 - Pode ser ajustado para tarefas específicas através de fine-tuning (exemplos de entrada e saída)

Modelos de Linguagem Pré-Treinados

- Decoder-only
 - Possuem capacidade de gerar texto
 - LLMs mais conhecidas utilizam esta estratégia por facilidade de treinamento (mais escalável)
 - GPT
 - Claude
 - Gemini

Modelos de Linguagem Pré-Treinados

- Encoder-Decoder
 - Estratégia utilizada para mapeamento de sequências de tamanhos diferentes
 - As tarefas incluem tradução, sumarização e reconhecimento de texto
 - Modelos incluem Text-to-Text Transfer Transformer (T5) e Bidirectional and Auto-Regressive Transformers

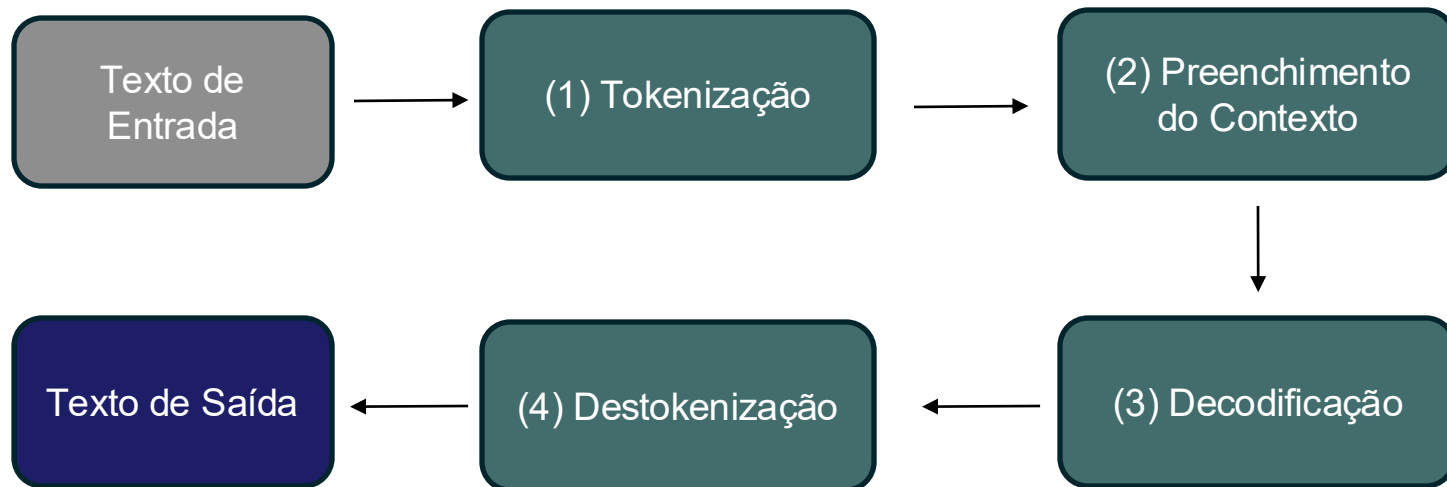
Exercício para a Próxima Aula

- Buscar um artigo que utilizou um modelo de linguagem baseado na arquitetura Transformer para a resolução de um problema de seu interesse
- Apresentar para a turma o nome do modelo utilizado, tipo (encoder-only, decoder-only ou encoder-decoder), quantidade de parâmetros e o problema solucionado
- Os resultados foram positivos?
- Quais as limitações encontradas?

O que é GPT?

- GPT-1 foi o primeiro modelo baseado em Transformer treinado para geração de texto
- Em português: *Transformer* Pré-treinado Generativo
- Generativo: Capacidade de gerar texto
- Pré-treinado: Previamente treinado com grande quantidade de dados
- Transformer: Arquitetura de Rede Neural

Fluxo Principal



Fluxo Principal

- (1) O texto de entrada é dividido em partes menores (tokens)
- (2) Cada parte é convertida para um vetor numérico (embeddings)
- (2) Os vetores são enviados ao modelo

Fluxo Principal

- (2) O modelo captura relações entre os tokens
- (3) O modelo prevê os tokens de continuação, um de cada vez
- (4) A lista de tokens gerados é convertida para texto (saída)

(1) Tokenização

- Divisão do texto de entrada em partes menores
- Estratégias comuns
 - Divisão por caractere: ['O', ' ', 'g', 'a', 't', 'o', ...]
 - Divisão por palavra: ['O', 'gato', ...]
- Por que é fundamental aplicar a tokenização?
- Qual é a melhor estratégia de divisão?

(1) Tokenização

- Por que é fundamental aplicar a tokenização?
- A divisão é fundamental para permitir ao modelo compreender o contexto entre diferentes partes do texto
- O modelo precisa compreender todo o texto
 - Mas sem enxergá-lo como um bloco único

(1) Tokenização

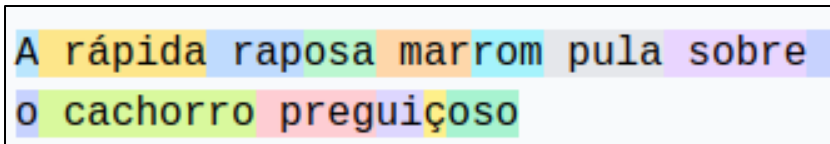
- Qual é a melhor estratégia de divisão?
- Separar em caracteres individuais faz com que cada token não tenha um significado inicial
- Separar em palavras dificulta a criação de vocabulário
 - E se o usuário enviar uma palavra incorreta ou pouco utilizada?

(1) Tokenização

- Na prática, cria-se um vocabulário mais versátil que combina caracteres com palavras
- Para isso, utiliza-se estratégias baseadas na análise de um grande volume de texto
- Diferentes algoritmos podem ser utilizados
 - Byte-pair encoding (BPE)
 - Unigram language modeling (ULM)

(1) Tokenização

- Objetivo da tokenização
 - Converter um texto de entrada em uma lista de tokens
- Exemplo (Tokenizador do gpt-4o)



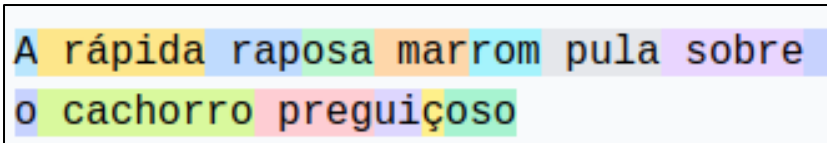
A rápida raposa marrom pula sobre
o cachorro preguiçoso

Entrada: [32, 54043, 6257, 6935, 2674, 511, 79334, 4844,
293, 168983, 20903, 1866, 704, 12408]

(1) Tokenização

- Comparação entre diferentes idiomas
 - Qual a diferença entre as tokenizações?
 - A divisão da palavra 'preguiçoso' é ideal?

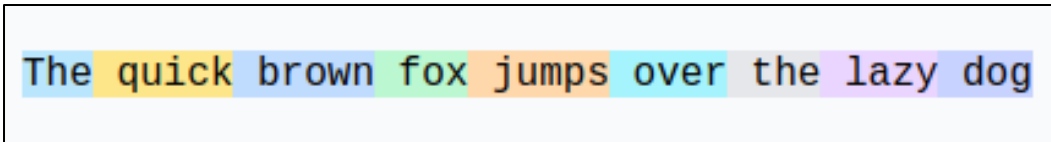
- Português



A rápida raposa marrom pula sobre
o cachorro preguiçoso

The image shows the Portuguese sentence "A rápida raposa marrom pula sobre o cachorro preguiçoso" with each word highlighted in a different color to represent tokens. The words are: A (yellow), rápida (blue), raposa (green), marrom (orange), pula (cyan), sobre (purple), o (light blue), cachorro (pink), and preguiçoso (teal).

- Inglês



The quick brown fox jumps over the lazy dog

The image shows the English sentence "The quick brown fox jumps over the lazy dog" with each word highlighted in a different color to represent tokens. The words are: The (yellow), quick (blue), brown (green), fox (orange), jumps (cyan), over (purple), the (light blue), lazy (pink), and dog (teal).

(1) Tokenização

- O tokenizador é treinado através de uma grande quantidade de texto (independente do modelo)
 - Como os dados são dominados pela língua inglesa, o resultado é mais satisfatório nesta língua
 - Em outras línguas, as palavras tendem a ser separadas em tokens menores

(1) Tokenização

- O tokenizador é treinado através de uma grande quantidade de texto (independente do modelo)
 - Desta forma, diferentes problemas podem surgir
 - Menor representação de significado
 - Contextos mais longos
 - Maior custo de treinamento do transformer

(1) Tokenização

- Exemplo Prático (Modelo DeepSeek-Coder)
(GUO, 2024)
 - Dados para treinamento do transformer
 - 87% de código fonte
 - 10% de texto em inglês (relacionado com código)
 - 03% de texto em chinês

(1) Tokenização

- Exemplo Prático (Modelo DeepSeek-Coder)
(GUO, 2024)
 - Dados para treinamento do tokenizador
 - Subconjunto dos dados para treinamento do transformer
 - Utilizou o algoritmo Byte-Pair Encoding
 - Vocabulário final com 32 mil tokens

Byte-Pair Encoding (BPE)

função BPE(texto T, iterações k) retorna vocabulário V

$V \leftarrow$ caracteres únicos em T

para $i = 1$ até k faça

$\text{tokenE}, \text{tokenD} \leftarrow$ par mais frequente de token adjacentes

$\text{novoToken} \leftarrow \text{tokenE} + \text{tokenD}$

$V \leftarrow V + \text{novoToken}$

 Atualiza cada ocorrência de tokenE, TokenD em T por
 novoToken

retorna V

Byte-Pair Encoding (BPE)

- Exemplo:
- Texto = A B C A B C A B
- Vocabulário = {A, B, C}
- Qual é o par mais frequente de tokens adjacentes?

Byte-Pair Encoding (BPE)

- Exemplo:
- Texto = A B C A B C A B
- Vocabulário = {A, B, C}
- TokenE, tokenD = (A, B)
- Novo Vocabulário = {A, B, C, AB}
- Texto Atualizado = AB C AB C AB

Byte-Pair Encoding (BPE)

- Próxima Iteração:
- Texto = AB C AB C AB
- Vocabulário = {A, B, C, AB}
- Qual é o par mais frequente de tokens adjacentes?
- TokenE, tokenD = (AB, C)
- Novo Vocabulário = {A, B, C, AB, ABC}
- Texto Atualizado = ABC ABC AB

Byte-Pair Encoding (BPE)

- Na prática, opta-se normalmente por separar as palavras em sub-palavras e tokens que combinam palavras não são permitidos
- Para isso, o texto é primeiro separado em espaços e pontuações com expressões regulares
- Com isso, o algoritmo BPE é ajustado para esse cenário

Byte-Pair Encoding (BPE) Adaptado

<u>Quantidade</u>	<u>Palavras</u>	<u>Vocabulário</u>
2	· n e w	{·, e, n, r, s, t, w}
2	· r e n e w	
1	s e t	
1	· r e s e t	

Qual é o par mais frequente de tokens adjacentes?

Byte-Pair Encoding (BPE) Adaptado

<u>Quantidade</u>	<u>Palavras</u>	<u>Vocabulário</u>
2	· ne w	{·, e, n, r, s, t, w, ne}
2	· r e ne w	
1	s e t	
1	· r e s e t	

Qual é o par mais frequente de tokens adjacentes?

Byte-Pair Encoding (BPE) Adaptado

<u>Quantidade</u>	<u>Palavras</u>	<u>Vocabulário</u>
2	· new	{·, e, n, r, s, t, w, ne, new}
2	· r e new	
1	s e t	
1	· r e s e t	

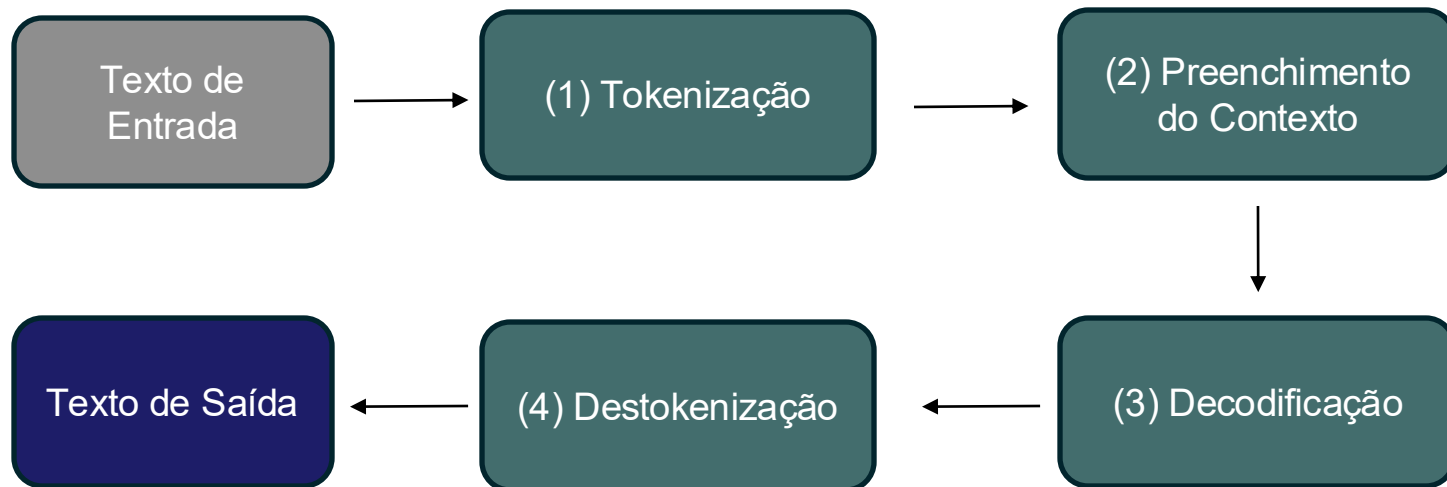
Qual é o par mais frequente de tokens adjacentes?

Byte-Pair Encoding (BPE) Adaptado

<u>Quantidade</u>	<u>Palavras</u>	<u>Vocabulário</u>
2	· new	{·, e, n, r, s, t, w, ne, new, ·r}
2	·r e new	
1	s e t	
1	·r e s e t	

Qual é o par mais frequente de tokens adjacentes?

Fluxo Principal



(2) Preenchimento do Contexto

→ Embeddings

- Os tokens são transformados em vetores numéricos para que o modelo consiga compreender a informação
- Essa transformação é armazenada em uma matriz e aprendida durante o treinamento da arquitetura transformer

1	Token ID		Dimension 0		Dimension 1		Dimension 2
2	-----						
3	0 (pad)		0.0		0.0		0.0
4	1 (hello)		0.23		-0.45		0.78
5	2 (world)		0.12		-0.33		0.89
6	3 (cat)		0.67		0.21		-0.12
7	...		...		...		...
8	50256		-0.34		0.56		0.23

(AKAN, 2025)

(2) Preenchimento do Contexto

- Por que não enviar diretamente a lista de Token IDs?

1	Token ID		Dimension 0		Dimension 1		Dimension 2
2	-----						
3	0 (pad)		0.0		0.0		0.0
4	1 (hello)		0.23		-0.45		0.78
5	2 (world)		0.12		-0.33		0.89
6	3 (cat)		0.67		0.21		-0.12
7	...		...		...		...
8	50256		-0.34		0.56		0.23

(AKAN, 2025)

(2) Preenchimento do Contexto

- Por que não enviar diretamente a lista de Token IDs?
 - A rede neural vai aprender relações incoerentes entre tokens
 - Com a utilização de embeddings, relações satisfatórias são aprendidas pelo próprio modelo

Exemplo clássico: rei + mulher - homem $\approx$ rainha

(2) Preenchimento do Contexto

- Exemplos Reais
 - BERT-base: 30.522 tokens x 768 dimensões
 - Llama-7b: 32.000 tokens x 4.096 dimensões
 - GPT-3: 50.257 tokens x 12.288 dimensões
 - └─ ~615 milhões de 175 bilhões de parâmetros

Exercício para a Próxima Aula

- Buscar um artigo que utilizou um modelo de linguagem baseado na arquitetura Transformer para a resolução de um problema de seu interesse
- Apresentar para a turma o nome do modelo utilizado, tipo (encoder-only, decoder-only ou encoder-decoder), quantidade de parâmetros e o problema solucionado
- Os resultados foram positivos?
- Quais as limitações encontradas?

Referências

- JURAFSKY, Daniel; MARTIN, James H. **Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models.** 3. ed. Stanford: Stanford University, 2026.
- VASWANI, Ashish et al. **Attention is all you need.** Advances in neural information processing systems, v. 30, 2017.
- KLEINA, Nilton Cesar Monastier. **Rumo ao bi: ChatGPT chega a 900 milhões de usuários ativos por semana.** TecMundo, 2026.
- BAHDANAU, Dzmitry; CHO, Kyunghyun; BENGIO, Yoshua. **Neural machine translation by jointly learning to align and translate.** arXiv preprint arXiv:1409.0473, 2014.

Referências

- FERRER, Josep. **How Transformers Work: a detailed exploration of transformer architecture**. DataCamp, 2026.
- BENTOML. **How does LLM inference work?** LLM Inference Handbook, 2025.
- MINAEE, Shervin et al. **Large language models: A survey**. arXiv preprint arXiv:2402.06196, 2024.
- NOBLE, Joshua. **Encoder-decoder model**. IBM, 2025.

Referências

- GUO, Daya et al. **DeepSeek-Coder**: when the large language model meets programming--the rise of code intelligence. arXiv preprint arXiv:2401.14196, 2024.
- AKAN, Oz. **Words, Tokens and Embeddings**. Luminary Blog, 2025.

Aula 1

Grandes Modelos de Linguagem

Gabriel Lenz Balatka

Orientador: Rafael Stubs Parpinelli

Joinville, 10 de junho de 2026



UDESC
UNIVERSIDADE
DO ESTADO DE
SANTA CATARINA

LABICOM
Laboratório de Pesquisa
em Inteligência Computacional